Affine-layer batch tree: report

Question and scope

Can point additions for 128 independent fixed-radix-14 secp256k1 outputs be arranged as per-scalar balanced affine trees, pooling denominators into one batch inversion at each tree level, instead of using XYZZ accumulators and a final batch recovery? The table, recoder, and 15-leaf schedule are unchanged. A full candidate batch uses at most four shared inversion rounds. `AffineLayer.lean` proves the tree-level sum identity under explicit pair-addition hypotheses; `BatchInverse.prefix_reverse_identity` in the parent experiment proves the batch inverse witness. The concrete Rust field/curve refinement remains supported by native replay rather than a Lean model.

Three retained panels

Each panel contains 96 verified arms and 48 paired comparisons on the same frozen 128-scalar fixture. The first two CPU 13 panels remain separate because their predeclared A/A gates failed. The third panel is a new CPU 40 run with only the execution pin changed in its new rules file; it passes all A/A gates and qualifies for the Tier 1 instance.

Job	CPU	XYZZ mean online time	Affine-layer mean	XYZZ / affine median	Paired
`c24e37df66ad4b1a891f1805798a0887`	13	62.193067 ms	54.096080 ms	1.171280×	
`fe00f695389443f08d6cbf003f1a8b7e`	13	61.819930 ms	53.432215 ms	1.164518×	
`251bb8443cad471ab3e6162307fd0c46`	40	61.885156 ms	53.156432 ms	**1.169535×**	

The two CPU 13 comparison intervals are descriptive, not qualified: one repeat failed both controls at 1.286019 and 0.887931 in the first job; one post-control failed at 1.165546 in the second. The CPU 40 panel has no A/A failure, stable artifacts, and output checks on all arms. It measured 4.325881 µs per candidate output versus 5.036227 µs for XYZZ. Both paths retained 26,035,988 bytes; candidate table preparation averaged 3,957.984 ms and reference preparation 3,981.450 ms, outside the online interval. The ratio is a batch-throughput result for this 128-output workload, not one-call latency or proof of exclusive physical-host access.

Audits and failed attempts

All three committed `analyze_affine_tier1.py --job ...` runs reproduced their corresponding audit JSON byte-for-byte. The CPU 13 panel logs retain their noisy A/A outcomes; the later CPU 40 panel is a new independent record and does not overwrite them. Source, binary, fixture hashes, arm outputs, and raw measurements are checked by each audit. The README lists archive digests and exact rerun commands. Lean output is retained in `lean-check.log`; Lean is unavailable in this local runtime, so I did not claim a new local proof run.

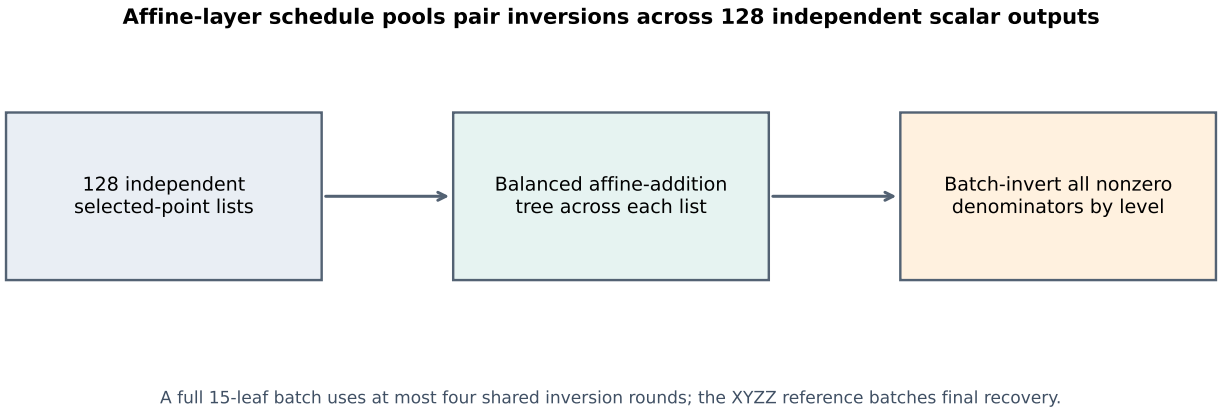
Graph and reproduction

The source-generated diagram shows how candidate inversion pooling crosses the 128 independent scalar trees. The panel graph keeps the two failed-gate intervals visible and marks them descriptive, while the CPU 40 result is the only qualified point. `visuals.py` reads the committed audit JSON files and emits the SVG/PDF figures. No end-to-end scalar-pipeline cost changed here: the broader experiment remains a stage-level fixed-base scalar diagnostic, and the inspected parent experiment has no canonical pipeline graph that this measurement updates. The report graph is the appropriate local view; it does not promote the stage result into an end-to-end claim.

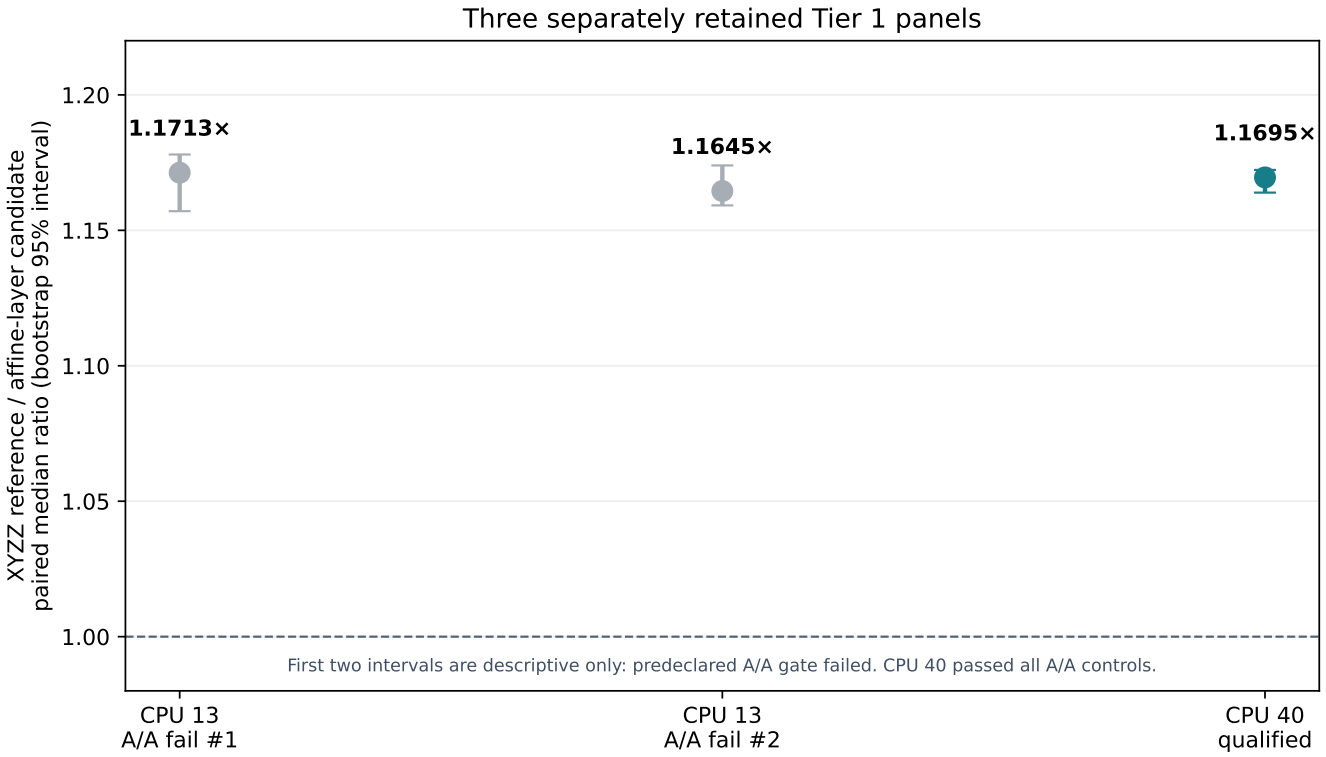
Rebuild with:

```
python3 experiments/prime-j0-affine-layer-batch-20261015/visuals.py
pandoc experiments/prime-j0-affine-layer-batch-20261015/REPORT-2026-10-19.md \
  --from=gfm --pdf-engine=xelatex -V geometry:margin=0.75in \
  -o experiments/prime-j0-affine-layer-batch-20261015/REPORT-2026-10-19.pdf
```

Schedule diagram (SVG) · PDF



Panel comparison graph (SVG) · PDF



Decision and next step

The CPU 40 panel qualifies the affine-layer schedule for the frozen 128-output batch workload. Preserve the earlier noise-gate failures as part of the record. The next useful test is to measure this schedule under additional scalar input laws and separately evaluate cold end-to-end cost before treating it as a general scalar-multiplication improvement.